

Salesforce B2C Commerce API (SCAPI) Client

Storefront Next includes a type-safe API client for calling Salesforce B2C Commerce Shopper APIs from your storefront. This client provides full TypeScript support with autocomplete, type checking, and semantic operation names.

What Is SCAPI?

The B2C Commerce API (SCAPI) is a collection of RESTful APIs for interacting with B2C Commerce instances. Also known as Salesforce Commerce API or simply Commerce API, it provides endpoints for products, search, baskets, customers, orders, and so on.

The SCAPI client in `@salesforce/storefront-next-runtime/scapi` wraps these APIs with:

- Minimal bundle size: Built on [openapi-fetch](#), a lightweight fetch client with near-zero runtime overhead.
- Type-safe operations: Full TypeScript inference from OpenAPI specifications.
- Semantic method names: Call `getProduct()` instead of `GET /products/{id}`.
- Automatic parameter injection: Common parameters like `organizationId` and `siteId` are handled for you.
- Built-in authentication helpers: Guest login, registered login, social login, passwordless login, token refresh, and password reset.

Creating API Clients

Create API clients that use `createCommerceApiClient`.

```
1 import { createCommerceApiClient } from "@salesforce/s
2
3 const clients = createCommerceApiClient({
4   baseUrl: "https://shortcode.api.commercecloud.salesfc
5   organizationId: "f_ecom_xxx",
6   siteId: "RefArch",
7   clientId: "your-slas-client-id",
8   redirectUri: "https://yoursite.com/callback",
9   locale: "en-US", // Optional
10  clientSecret: process.env.COMMERCE_API_SLAS_SECRET, /
11 });
```

Contents

What Is SCAPI?

Creating API Clients

Authentication

Fetching Data

Mutations

Available API Clients

TypeScript Types

Error Handling

Response Format

Middleware

Custom SCAPI Endpoints

See Also



<code>baseUrl</code>	Yes	Commerce API base URL
<code>organizationId</code>	Yes	Your organization ID (automatically added to path parameters)
<code>siteId</code>	Yes	Your site ID (automatically added to query parameters)
<code>clientId</code>	Yes	Shopper Login and Session API (SLAS) client ID
<code>redirectUri</code>	Yes	OAuth redirect URI
<code>locale</code>	No	Locale for requests (for example, <code>en-US</code>)
<code>clientSecret</code>	No	SLAS client secret. See Public vs Private SLAS Client .

Public vs Private SLAS Client

The `clientSecret` parameter determines which SLAS authentication flow the client uses.

- Public client (no secret): When `clientSecret` is omitted, the auth helpers use the public client flow with Proof Key for Code Exchange (PKCE). This behavior is suitable for client-side apps where the secret can't be kept secure.
- Private client (with secret): When `clientSecret` is provided, the auth helpers use the private client flow. This behavior enables extra features, such as passwordless login, and is suitable for server-side apps where the secret can be kept secure.

Authentication

Before making API calls, you must get an access token. The client includes an `auth` namespace with helper functions for SLAS shopper login and session API operations.

These authentication helpers are intentionally **stateless**—they don't store tokens, session data, or any intermediate state internally. Each function call is independent, giving you full control over how and where you store credentials. This design ensures compatibility with server-side rendering, avoids shared state issues across requests, and lets you choose your preferred storage strategy, such as cookies, session storage, database, and so on.

Guest Login





Registered User Login

```
</> | [icon]
1 const tokens = await clients.auth.loginWithCredentials({
2   username: "user@example.com",
3   password: "password123",
4 });
```

Token Refresh

```
</> | [icon]
1 const newTokens = await clients.auth.refreshToken({
2   refreshToken: storedRefreshToken,
3 });
```

Logout

```
</> | [icon]
1 await clients.auth.logout({
2   accessToken: currentAccessToken,
3   refreshToken: currentRefreshToken,
4 });
```

Social Login

```
</> | [icon]
1 // Step 1: Get authorization URL
2 const { url, codeVerifier } = await clients.auth.socialLogin({
3   hint: "google", // or 'facebook', 'apple', etc.
4 });
5
6 // Store codeVerifier securely, then redirect user to url
7
8 // Step 2: Exchange code (after user returns)
9 const tokens = await clients.auth.socialLogin.exchangeCode({
10   code: authorizationCode,
11   codeVerifier: storedCodeVerifier,
12   redirectUri: "https://yoursite.com/callback",
13 });
```

Adding the Access Token to Requests

After you have an access token, add it to API requests by using middleware. Note that SLAS authentication endpoints handle their own authorization and must be skipped.

```
</> | [icon]
1 import { SLAS_AUTH_ENDPOINTS } from "@salesforce/storefront-auth";
2
3 clients.use({
4   onRequest({ request }) {
```



```
10 }
11
12     request.headers.set("Authorization", `Bearer ${accessToken}`);
13     return request;
14 },
15 });
```

Fetching Data

The client automatically includes `organizationId` and `siteId` in all requests, so you only provide operation-specific parameters.

Fetching a Product

```
</> | [icon]
1  const { data: product } = await clients.shopperProducts.fetch({
2    params: {
3      path: { id: "product-123" },
4      query: {
5        expand: ["availability", "prices", "variations",
6          allImages: true,
7      },
8    },
9  });
```

Search Products

```
</> | [icon]
1  const { data: searchResults } = await clients.shopperSearch.fetch({
2    params: {
3      query: {
4        q: "shoes",
5        limit: 24,
6        offset: 0,
7        sort: "best-matches",
8        expand: ["prices", "images", "variations"],
9      },
10    },
11  });
```

Fetching Categories

```
</> | [icon]
1  const { data: category } = await clients.shopperProductCategories.fetch({
2    params: {
3      path: { id: "mens-clothing" },
4      query: { levels: 2 },
5    },
6  });
```

Mutations

[Try for free](#)

```
1 const { data: updatedBasket } = await clients.shopperBasket.updateBasketItem({
2   params: {
3     path: { basketId: "basket-123" },
4   },
5   body: [
6     {
7       productId: "product-456",
8       quantity: 2,
9     },
10  ],
11 });
```

Updating a Basket Item

```
1 const { data: updatedBasket } = await clients.shopperBasket.updateBasketItem({
2   params: {
3     path: {
4       basketId: "basket-123",
5       itemId: "item-789",
6     },
7   },
8   body: {
9     quantity: 3,
10  },
11 });
```

Available API Clients

The client provides access to all SCAPI endpoints.

Client	Description
shopperAvailability	Product availability and inventory
shopperProducts	Products, categories, and catalog data
shopperSearch	Product search and suggestions
shopperBasketsV1	Basket operations (legacy)
shopperBasketsV2	Basket operations (current)
shopperCustomers	Customer accounts and profiles
shopperOrders	Order history and details
shopperLogin	SLAS authentication



shopperContext	Shopper session context
shopperStores	Store locator
shopperExperience	Page Designer content
shopperGiftCertificates	Gift certificate operations
shopperSeo	SEO metadata
shopperConsents	Customer consent management
auth	Authentication helpers. See Authentication .

TypeScript Types

The client exports TypeScript types for all API schemas and operations.

```
1 import type { ShopperProducts, ShopperSearch } from "@salesforce/storefront-next"
2
3 // Use schema types for data
4 type Product = ShopperProducts.schemas["Product"];
5 type Category = ShopperProducts.schemas["Category"];
6 type SearchResult = ShopperSearch.schemas["ProductSearchResult"];
7
8 // Use operation types for parameters
9 type SearchParams = ShopperSearch.operations["productSearch"];
```

Error Handling

API errors throw an `ApiError` with detailed information:

```
1 import { ApiError } from "@salesforce/storefront-next"
2
3 try {
4   const { data } = await clients.shopperProducts.getProduct
5 } catch (error) {
6   if (error instanceof ApiError) {
7     console.error("Status:", error.status); // 404
8     console.error("Message:", error.body.detail); // "Product not found"
9     console.error("URL:", error.url); // Request URL
10    console.error("Method:", error.method); // "GET"
11  }
12 }
```

ApiError Properties

[Try for free](#)

statusText	string	HTTP status message
body	ErrorDetail	Parsed error response
rawBody	string	Raw response text
headers	Headers	Response headers
url	string	Request URL
method	string	HTTP method

Response Format

All operations return both the parsed data and the raw response.



```
1 const { data, response } = await clients.shopperProduct
2
3 // Typed data from the API
4 console.log(data.name);
5 console.log(data.price);
6
7 // Raw response for headers, status, etc.
8 console.log(response.headers.get("etag"));
9 console.log(response.status);
```

Middleware

Add custom behavior to requests and responses:



```
1 import type { Middleware } from "@salesforce/storefront
2
3 const loggingMiddleware: Middleware = {
4   onRequest({ request }) {
5     console.log(`[API] ${request.method} ${request.url}`);
6     return request;
7   },
8   onResponse({ response }) {
9     console.log(`[API] ${response.status}`);
10    return response;
11  },
12 };
13
14 // Add to all clients
15 clients.use(loggingMiddleware);
16
17 // Or add to a specific client
18 clients.shopperProducts.use(loggingMiddleware);
```

[Try for free](#)

currently supported by the client. To call custom SCAPI endpoints, use the native `fetch` API or another HTTP client.

See Also

- [B2C Commerce API \(SCAPI\)](#) - Official SCAPI documentation
- [openapi-fetch](#) - The underlying fetch client library

DID THIS ARTICLE SOLVE YOUR ISSUE?

Let us know so we can improve!

[Share your feedback](#)

DEVELOPER CENTERS

[Heroku](#)
[MuleSoft](#)
[Tableau](#)
[Commerce Cloud](#)
[Lightning Design System](#)
[Einstein](#)
[Quip](#)

POPULAR RESOURCES

[Documentation](#)
[Component Library](#)
[APIs](#)
[Trailhead](#)
[Sample Apps](#)
[AppExchange](#)

COMMUNITY

[Trailblazer Community](#)
[Events and Calendar](#)
[Partner Community](#)
[Blog](#)
[Salesforce Admins](#)
[Salesforce Architects](#)

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#) [Use of Cookies](#)

[Cookie Preferences](#)



[Your Privacy Choices](#)

